

Key questions to ask yourself

Synthesis document for the big project (distributed MapReduce system on the school machines). It gathers the key questions from the labs and from the paper *MapReduce: Simplified Data Processing on Large Clusters* (Dean & Ghemawat, OSDI'04). These are not exam questions: they are the questions **your group** must ask itself at every step to avoid heading in the wrong direction.

0. Working method and group organization

- How do you split the work within the group? Who is responsible for what (deployment, protocol, map/reduce, metrics, fault tolerance)?
- Is anyone in the group already working on **Kafka Streams**, so that it is not left until the very last minute?
- How is the **atmosphere and the collaboration** inside the group? Does everyone feel involved and able to contribute?
- How do you share your findings and your progress (Git repository, shared notes)?
- Does everyone in the group know how to deploy, run and clean the system, or does a single member master everything?
- Did you formalize your protocol on <https://www.sequencediagram.org> **before** coding, or are you coding "blindly"?
- Do you all have a working GitHub Copilot license and a working VSCode?

1. Machine discovery and deployment

- Which `tp.telecom-paris.fr` API do you use? Did you spot the request the browser sends to `https://tp.telecom-paris.fr/ajax.php` to get the raw JSON?
- How do you tell a **live** machine from a powered-off or unreachable one?
- Do you use the **canonical names** (e.g. `tp-1a201-05.enst.fr`, ending in `.enst.fr`)?
- Does your deployment script use **a single SCP** (to the HOME, which is on the NFS) then **100 SSH** to start the servers? Why is this much better than 100 SCP?
- How do you choose the **listening port**? What happens if the port is already used by someone else?
- Does your **cleaning** script really kill **all** the deployed servers? Can you cleanly redeploy/restart after updating the server?
- What happens if **a machine does not answer** during deployment? Does your script block, or carry on (timeout)?
- Did you configure SSH with **keys** (public/private) and **fingerprint bypass** so you don't type a password or a "yes" at every connection?

- Looking back, what was the **hardest part** of getting deployment to work? Did you hit blockers (network, SSH, ports, NFS, unreachable machines), and how did you get past them?

2. NFS vs local disk (critical point)

- Do you know **where** you actually write when you write into your HOME? (answer: on the **NFS**, not the local disk of the machine)
- Where do you write the **map phase intermediate files**? In Figure 1 of the paper they are written to the **LOCAL disk**, not the NFS.
- Concretely, how do you write **locally** on the school machines (e.g. /tmp)?
- Beyond /tmp, are there **other local disks or locations** available on the machines? How do you **explore** a machine to find them (mounted filesystems, scratch/local partitions, df, mount)? This is something you have to investigate yourselves.
- Why is writing locally important both for **performance** and to avoid **overloading the NFS** ("the poor little server")?

3. Protocol design (Main / Workers)

- What exactly are the roles of the **Main** and the **Workers** (formerly Master/Workers)?
- How does the Main **assign** map tasks and reduce tasks to the available nodes?
- How do you **synchronize** the map → shuffle → reduce phases? How does the Main know a phase is finished before starting the next one?
- Do you have a **space-time diagram** of the V1 protocol, **without fault tolerance** (KISS principle: *Keep It Simple and Stupid*)?
- How do you **bootstrap** the system (the chicken-and-egg problem)? Does it require an improved version of the protocol?
- Which **signals/messages** do you exchange between Main and Workers, and in which **format**?

4. MapReduce core (word frequency first)

- For word frequency: what exactly does your **map** produce? what does your **reduce** do?
- How do you decide **which node** is responsible for **which key** in the reduce? (formula: $\text{hash}(\text{key}) \bmod \text{number_of_nodes}$)
- What does this formula **guarantee**? What happens if the number of nodes changes along the way?
- How does the reduce phase **read** the intermediate data written **locally** on the map nodes (the famous *remote read*)?
- Are your results **correct**? How do you check them (sanity check: compare the number of pairs in and out, cross-check against a reference computation)?

- How do you **validate** the output of the whole distributed system? Can you, for example, run a simple **single-machine** word count on the same small dataset and confirm it gives **exactly the same counts**?

5. The data: Common Crawl

- How do you read the splits? (some are on a local server when you mount the /cal/commoncrawl folder)
- Do you know the **structure** of Common Crawl (file formats, organization)?
- Did you first test on **small splits** (proof of concept, a few words) before moving to real splits, then 100, 1000 or more?
- Can you read **directly from Amazon's servers** rather than through the internal NFS? Why is it preferable (the school NFS is slow and overloaded when everyone reads hundreds of files at the same time)?
- Why is Common Crawl worth knowing about (one of the main data sources for training AI today)?

6. Performance and Amdahl's law

- Do you time **every step**: deployment, cleaning, reading/writing files, synchronization/waiting, networking/communication, computing (map/reduce)?
- Where is the **bottleneck**? Can you **prove** it with measurements, and not just assume it?
- Do you have a **reference point** for Amdahl's law (speedup = 1, number of nodes = 1)?
- When you plot **speedup against the number of nodes**, do you use **exactly the same dataset** at every point of the graph? (otherwise the graph is **WRONG**)
- One **exception**: on a **single machine** only, you may *project/interpolate* under a strong linear-scaling hypothesis — e.g. if 100 MB takes 1 minute on one machine, you *may* assume 1000 MB takes ~10 minutes. But **as soon as you have two or more nodes**, this assumption breaks down: you can no longer extrapolate, you must **measure**.
- What does Amdahl's law **predict** for your system? What is the incompressible **sequential part**?
- Did you test several scales (a few splits → 100 → 1000+) to observe the real behavior?

7. Fault tolerance

- How do you **detect** that a worker is down (ping / timeout)?
- What does the Main do when a worker dies? (re-execution of the affected tasks)
- If a dead **map** worker had already written its intermediates locally, is that data lost? Must the task be redone?
- Do you handle **stragglers** (abnormally slow machines)? (the *backup tasks* idea from the paper)

- Are your output writes **atomic** (write to a temporary file then rename) to avoid duplicates / corrupted results?
- Can you **demonstrate** fault tolerance by deliberately killing nodes mid-computation (as in the Google paper)?
- What happens if the **Main** itself goes down?

8. Comparison and Kafka Streams (light — little time)

Kafka is secondary: you have little time to test it. The core is still **your** system.

- Can you situate the **batch** approach (your system, Hadoop) vs the **stream** approach (Kafka Streams, incremental output that keeps updating continuously)?
- Did you get a **minimal wordcount** working with Kafka Streams, starting from the downloaded files (no Docker, no root rights on the school machines)?
- For **Hadoop** (a Java implementation), you are **not** asked to run it: find **documentation** and compare it with your system on paper. For example, you almost certainly did **not** implement **HDFS** — and there are many other things a full system like Hadoop provides that your **deliberately simplified** version does not. Which ones can you identify?
- How do you compare the three on **results, speed and implementation difficulty** (your system vs Hadoop vs Kafka Streams)?
- Lead (without going into detail): with Kafka, could you read Common Crawl **directly**, instead of through the overloaded NFS, by imagining a **Kafka source** or a **custom connector**?

9. Use cases, report and demo

- Which **3 use cases** beyond word frequency? (ideas: languages, popularity, ranking, size, density...) Which **question** do you ask the dataset?
- Are the results of each use case **interpretable** and explained?
- Does your **written report** properly cover: architecture, protocol design, performance metrics, Amdahl's law graph, pain points (what broke and why), the 3 use cases, and the comparison (batch vs stream)?
- Is your **demo** ready (10 min presentation + 10 min questions) and is the work well distributed between members?